

W1 INTRO/ASYMPTOTICS

DSA goal: represent data so operations meet required asymptotic bounds; exam answers need **algorithm + correctness + complexity**. State input size (n, m, h, k) , model assumptions, and whether worst/expected/amortised.
O; upper bound: $1 < \Theta(n) \leq O(g) \text{ iff } \exists c, n_0 : f(n) \leq cg(n) \text{ for } n \geq n_0$.
Growth order: $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$.
Constants/lower terms vanish; bases of logs constant factor.
Loop count: arithmetic loop $\sum_{i=1}^n 1 = \Theta(n)$; nested triangular $\sum_{i=1}^n \sum_{j=1}^i 1 = \Theta(n^2)$; halving/doubling loop $\Theta(\log n)$; loop body dependent on n must be summed.
Recurrences: $T(n) = aT(n/b) + f(n)$; compare $n^{\log_b a}$ vs f .
 $T(n) = T(n/2) + O(1) = O(\log n)$; $T(n) = T(n/2) + O(n)$; $2T(n/2) + O(n) = O(n \log n)$; $4T(n/4) + O(n) = O(n^2)$.
Proof patterns: loop invariant (init, maintain, terminate), induction over size/height, exchange argument for greedy, cut/cycle property for MST, divide-and-conquer decomposition, contradiction/counterexample for false greedy.
Pseudo-code style: precise enough to implement; name DS ops and their cost; avoid language-specific syntax if irrelevant. If using sort, heap, BFS/Dijkstra, cite known cost.
Exam traps: $O(n)$ algorithm not guaranteed faster on every finite input than $O(n \log n)$; has expected vs worst-case differs; output size may dominate.

Pattern	Count	Cost
for $i = c$ to n/c	$\Theta(n)$	
while $i \leq 2 \log n$	$\Theta(\log n)$	
all pairs	$n(n-1)/2 = \Theta(n^2)$	
sort/scan	$n \log n + n = \Theta(n \log n)$	

Correctness wording: "Invariant after processing first i items: data structure stores exactly $\dots$ ". At termination $i = n$, invariant implies desired output.
Lower bound reminder: comparison sorting needs $\Omega(n \log n)$ comparisons in worst case; non-comparison sorts exploit bounded alphabet/range/word size.

W2 ADT/LISTS

ADT: specifies operations and semantics, not representation. Implementation choice controls cost: array list, singly/doubly linked list, positional list, map, stack, queue.

Op	Array	Single LL	Double LL
access i	$O(1)$	$O(1)$	$O(1)$
insert front	$O(n)$	$O(1)$	$O(1)$
append	amort $O(1)$	$O(1)$ tail	$O(1)$
remove node	$O(n)$	need prev	$O(1)$

Array list: contiguous, cache-friendly, random access $O(1)$; insert/delete middle shifts $O(n)$; dynamic resize: single resize $O(n)$ but append amortised $O(1)$.
Linked list: stores references; no random access; insertion/removal after known node $O(1)$; more memory overhead; good for frequent local splicing.
Position: abstract handle to node, hides implementation. Validate position belongs to list and is still live; sentinels simplify edge cases.
Reverse-order digit lists: traverse both heads with carry; append digit $(x+y+carry) \bmod 10$; carry $((x+y+carry)/10)$; stop when both null and carry 0. Cost $O(\max(m, n))$.
Sorted vs unsorted list map: unsorted put $O(1)$ if duplicates allowed/append but get/remove $O(n)$; sorted get by linear list still $O(n)$ but supports early stop; array sorted can binary search $O(\log n)$ but insertion $O(n)$.
Iterator mutation: if removing while scanning array, do not skip shifted element. For linked list maintain prev/current.
Space: list nodes $O(n)$, array capacity may be $O(n)$ with unused slack; doubling keeps total copied work $O(n)$ over n appends.
Use cases: queue via linked list with head/tail; stack via dynamic array or linked list; priority queue not ordinary queue: delete-min/max by key.
Invariant for list algorithms: maintain processed prefix result and unprocessed suffix pointer; every node visited once unless nested search.

W3 STACKS/QUEUES/COST

Stack: LIFO, ops push/pop/top in $O(1)$; array stack may resize. **Queue:** FIFO, enqueue rear/dequeue front $O(1)$ with circular array or linked head/tail.
Expression evaluation: two stacks: values and operators. Scan tokens; push nums/ops/ '(' / on ') ' pop op + operands, compute, push result. Fully parenthesised + , × expression cost $O(n)$.
Validity checking: parentheses/XML tags use stack of expected closers/open tags. On closing token, stack top must match; final stack empty. Correctness by nesting invariant.
Deque: insert/delete front/back $O(1)$ with doubly linked list or circular buffer; supports both stack and queue behaviour.
Amortised analysis: sequence cost / operations. Dynamic array doubling: although some pushes cost $O(n)$, total copying over n pushes $O(n)$, amortised $O(1)$.
Remove-even array list: if remove at index shifts left $O(n)$, repeated removals worst $\Theta(n^2)$; linked list with known prev can do $\Theta(n)$ scan.
Complexity from height/depth: tree path algorithm from node to root costs number of ancestors times per-node work. If $\text{bar}(u) = O(\log n)$ and path length h , cost $O(h \log n)$, worst $O(n \log n)$.
If algorithm calls $\text{bar}(u)$ on search path, cost is $\sum_{u \text{ path}} \text{cost}(\text{bar}(u))$, not simply $O(h)$.
ADT selection: Insert-large + delete-min can be queue if inserted keys strictly increasing; append at back, delete front $O(1)$. Heap/BST would be overkill.
MCQ checks: stack reverses order; queue preserves order; priority queue removes by priority, not arrival; circular array full/empty distinguished by size or spare slot.

W4 TREES/RECURSION

Tree terms: root, parent/child, sibling, ancestor/descendant, leaf, depth, height. For node u , subtree T_u , size $|T_u|$, tree height h can be $n-1$ unless balanced.
Traversals: preorder visit-root-left-right; inorder left-root-right (BST sorted); postorder left-right-root; level order BFS queue. All visit each node once: $O(n)$.
Recursive tree analysis: if one recursive call down a path, depth h ; if both children, recurrence sums over subtrees. If work at each node $O(1)$ then $O(n)$. If work $O(|T_u|)$ at every node, worst skew $\Theta(n^2)$, balanced $\Theta(n \log n)$.

W5 BST/AVL/BALANCED

Binary tree implementation: linked nodes with left/right/parent; array heap style only works compactly for complete trees. General/BST array layout wastes space under skew and expensive gaps.
BST property: all keys left $< u$, $key > u$ all keys right. Search/insert/delete follows one root-to-leaf path: $O(h)$. Inorder gives sorted sequence.
Build BST from preorder in $O(n)$: maintain index and allowed range (l, o, h) ; next key belongs if $l < o < h$; create node, recurse left (l, o, x) then right (x, h, h) . Each key consumed once.
Insert/delete in BST: if left child exists, max of left subtree; else climb until first ancestor for which node lies in right subtree. Search for predecessor of value x : track last key $< x$ while descending.
Deletion: leaf remove; one child splice; two children swap/copy predecessor/successor then delete that node. Cost $O(h)$.
BST best/avg if balanced: $O(\log n)$; worst skew: $O(n)$. State $O(h)$ first, then substitute.


W6 MAPS/HEAPS/SORT

Map: key-value dictionary supporting get/put/remove. Implementations: unsorted list, sorted array/list, hash table, balanced BST. Choice depends on ordered queries vs expected $O(1)$.

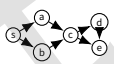
Impl	get	put	ordered?
unsorted list	$O(n)$	$O(1)$	no
sorted array	$O(\log n)$	$O(n)$	yes
AVL/RB	$O(\log n)$	$O(\log n)$	yes
hash eq	$O(1)$	$O(1)$	no

Priority queue: insert item with key, min/remove-min. Unsorted list: insert $O(1)$ remove-min $O(n)$; sorted list: insert $O(n)$ remove-min $O(1)$; heap: both $O(\log n)$.
Binary heap: complete binary tree with heap-order. Array index: parent $((i-1)/2)$, children $2i+1, 2i+2$ (0-index). Min-heap parent $\leq$ children.
Insert heap: append at end, up-heap/bubble up until heap-order restored; cost $O(n)$. **Delete-min:** replace root by last, down-heap with smaller child; $O(\log n)$.
Heapify: bottom-up downheap from last internal node to root builds heap in $O(n)$, not $O(n \log n)$. Heapsort: heapify + repeated remove $O(n \log n)$, in-place, not stable.
Sorting: selection/insertion worst $O(n^2)$; merge worst $O(n \log n)$ stable extra space $O(n)$; quicksort expected $O(n \log n)$ worst $O(n^2)$; heap sort stable extra space $O(n)$.
Comparison lower bound: any comparison sort worst $\Omega(n \log n)$; counting/radix can be linear if key universe/digit count bounded. Radix on 192-bit coordinates may be large constant/memory and not always practical.
kth smallest: keep max-heap of size k ; scan elements, replace root if new $< \text{root}$; root at end is kth smallest. Cost $O(n \log k)$, space $O(k)$.
Merge k sorted lists: min-heap of current heads; repeat extract-min and advance same list. Cost $O(N \log k)$ where $N = \sum m_i$.
Heap MCQ: array is valid min-heap iff every parent value $\leq$ both children; no sibling ordering required.

W7 HASHING/TRIES

Hash table: map key to bucket by $h(k)$. Separate chaining stores lists; open addressing probes array. Expected $O(1)$ if load factor controlled and hash distributes well; worst $O(n)$.
Load factor $\alpha = n/N$. Chaining expected successful search $O(1 + \alpha)$; keep $\alpha = O(1)$ by resizing.
Universal hashing: random choice from family reduces collision probability for any fixed key pair; still worst-case insertion sequence can make total cost high, but expected bounds improve.
Pigeonhole collision: if 10000 possible 5-digit/range values map to 100 buckets, some bucket has at least $\lceil 10000/100 \rceil = 100$ values.
Open addressing: linear probing clusters; quadratic/double hashing reduce primary clustering. Must handle deleted slots with tombstones or rehash. Load factor must stay < 1 .

W8 GRAPHS/TRaversal

Graph: $G = (V, E)$, $n = |V|$, $m = |E|$. Directed/undirected, weighted/unweighted, simple/multigraph. Adjacency list space $O(n + m)$; matrix $O(n^2)$, edge test $O(1)$.
BFS: queue, discover by layers from source; gives shortest path distances in unweighted graph. Cost $O(n + m)$ adjacency list. Invariant: when vertex dequeued, its distance is final.
DFS: recursion/stack; timestamps discover/finish; explores deeply. Used for cycle detection, topological order, connected components, SCC, articulation ideas. Cost $O(n + m)$.
BFS vs DFS: BFS good shortest unweighted path/layers; DFS good reachability, topological, backtracking. DFS order not unique without neighbour ordering.
DAG topological order: linear order with every edge $u \rightarrow v$ placing u before v . Exists iff directed graph acyclic. DFS reverse finish time or Kahn indegree queue. SCC: maximal set mutually reachable. Kosaraju: DFS finish order, reverse graph, DFS in decreasing finish; Tarjan uses lowlink stack; both $O(n + m)$.
Graph modification trick: reverse all directed edges to compute distances to target t by running single-source shortest path from t on reversed graph.
Traversal proof: discovered vertices partition into processed/frontier/unseen; each edge inspected once or twice. Parent pointers form BFS/DFS forest.


W9 SHORTEST/MST

Dijkstra: nonnegative weights. Maintain tentative dist, min-priority queue. Extract min u finalises $d[u]$; relax outgoing edges. Binary heap: $O(n + m \log n)$; Fibonacci: $O(m + n \log n)$.
Relax edge (u, v, w) : if $d[v] > d[u] + w$, set $d[v] = d[u] + w$, $\text{parent}[v]=u$, decrease-key.
Dijkstra fail: negative edges can make extracted distance nonfinal. Use Bellman-Ford for negative weights/no negative cycle $O(nm)$.
Feasible edge on $s-t$ path: compute $\text{dist}_s[x]$ from s and $\text{dist}_t[x]$ to t by reversed graph; edge (u, v) feasible under budget σ iff $\text{dist}_s[u] + w(u,v) + \text{dist}_t[v] \leq \sigma$. Scan edges for max weight.
Path via set S : multi-source Dijkstra from all $x \in S$ can compute nearest-set distances; for path $s \rightarrow S \rightarrow t$, combine $\text{dist}_s[x] + \text{dist}_t[x]$ or layered graph for exactly/all most one special vertex.
MST: connects all vertices of undirected weighted graph min total weight. Cut property: lightest edge crossing any cut is safe. Cycle property: heaviest edge on cycle not needed.
Kruskal: sort edges by weight; add if endpoints in different DSU components. Cost $O(m \log m)$; DSU nearly $O(\alpha(n))$.
Prim: grow tree from start; repeatedly add min edge from tree to outside. Heap adjacency list $O(m \log n)$; matrix $O(n^2)$.
Shortest path vs MST: shortest path between two vertices is not necessarily contained in MST; MST minimises total tree weight, not pairwise distances.
Answer style: for graph algorithm design, state representation, preprocessing, main algorithm, output, correctness invariant/cut/relaxation, cost in n, m .
Top final themes: Dijkstra with reversed graph, MST drawing, graph traversal order/layers, SCC/dominator variants. **Dijkstra proof:** let u be extracted min. Any alternative path to u via unsettled x has length at least $d[x] \geq d[u] + w$ nonnegative suffix, contradiction. Hence $d[u]$ final.
MST drawing: sort weights; Kruskal adds edges unless cycle. If equal weights, multiple MSTs possible; state one valid tree and total weight. If graph disconnected, MST does not exist; algorithm returns forest.
DSU: make-set each vertex; find with path compression; union by rank/size. Used in Kruskal and connectivity. Amortised almost constant $O(\alpha(n))$.
Shortest path tree vs MST: Dijkstra parent tree depends on source and preserves shortest distances from source only; MST has no source and minimises total edge weight.

W10 GREEDY

Greedy: make locally optimal choice and prove it can be extended to global optimum. Need greedy-choice property + optimal substructure; otherwise give counterexample.
Interval scheduling: sort by earliest finish; repeatedly choose compatible interval. Exchange proof: earliest finishing chosen interval leaves maximal remaining room. Cost $O(n \log n)$.

W5 BST/AVL/BALANCED

AVL: height-balanced BST with balance factor $bf = h_L - h_R$ at each node u . $bf \in \{-1, 0, 1\}$ at every node. Search $O(\log n)$; insert/delete may rotate.
AVL insertion: ordinary BST insert → update heights upward → first unbalanced node z with child y and grandchild x determines rotation: LL right-rotate, RR left-rotate, LR left on y then right on z , RL right on y then left on z .
AVL deletion: delete as BST then rebalance upward; unlike insertion, may need rotations at multiple ancestors.
Height update: $h(u) = 1 + \max(h_L, h_R)$; $bf_L = h_L - h_R$. Rotation preserves inorder, so BST property remains.

Case	Pattern	Fix	Notes
LL	left-left	rotate right	single
RR	right-right	rotate left	single
LR	left-right	left then right	double
RL	right-left	right then left	double

2-3/2-4 trees: all leaves same depth; internal node stores 1-2 (2-3) or 1-3 (2-4) keys. Search/insert/delete $O(\log n)$. Split overflow upward; merge/borrow underflow.
Red-black: encodes 2-4 tree; looser balance than AVL. AVL has smaller height, often faster lookup; RB fewer rotations for updates. Both asymptotic $O(\log n)$.
Order statistic augmentation: store subtree size at each node. For select/rank/median by size in $O(\log n)$; update sizes during rotations. For median $O(1)$ can keep pointers/s to middle keys and adjust on insert/delete.
Spay tree: self-adjusting; recently accessed keys move near root; amortised $O(\log n)$, good locality, no strict worst-case per operation.
B-tree: high branching factor for disk/cache; height small; choose when block access dominates.
Path validity MCQ: BST root-to-leaf sequence maintains shrinking interval: after going right lower bound increases, after going left upper bound decreases. Any later key must lie in current interval.
Proof: rotations are local transformations preserving inorder list; AVL height bound follows recurrence $N(h) = 1 + N(h-1) + N(h-2)$.

W6 MAPS/HEAPS/SORT

Map: key-value dictionary supporting get/put/remove. Implementations: unsorted list, sorted array/list, hash table, balanced BST. Choice depends on ordered queries vs expected $O(1)$.

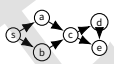
Impl	get	put	ordered?
unsorted list	$O(n)$	$O(1)$	no
sorted array	$O(\log n)$	$O(n)$	yes
AVL/RB	$O(\log n)$	$O(\log n)$	yes
hash eq	$O(1)$	$O(1)$	no

Priority queue: insert item with key, min/remove-min. Unsorted list: insert $O(1)$ remove-min $O(n)$; sorted list: insert $O(n)$ remove-min $O(1)$; heap: both $O(\log n)$.
Binary heap: complete binary tree with heap-order. Array index: parent $((i-1)/2)$, children $2i+1, 2i+2$ (0-index). Min-heap parent $\leq$ children.
Insert heap: append at end, up-heap/bubble up until heap-order restored; cost $O(n)$. **Delete-min:** replace root by last, down-heap with smaller child; $O(\log n)$.
Heapify: bottom-up downheap from last internal node to root builds heap in $O(n)$, not $O(n \log n)$. Heapsort: heapify + repeated remove $O(n \log n)$, in-place, not stable.
Sorting: selection/insertion worst $O(n^2)$; merge worst $O(n \log n)$ stable extra space $O(n)$; quicksort expected $O(n \log n)$ worst $O(n^2)$; heap sort stable extra space $O(n)$.
Comparison lower bound: any comparison sort worst $\Omega(n \log n)$; counting/radix can be linear if key universe/digit count bounded. Radix on 192-bit coordinates may be large constant/memory and not always practical.
kth smallest: keep max-heap of size k ; scan elements, replace root if new $< \text{root}$; root at end is kth smallest. Cost $O(n \log k)$, space $O(k)$.
Merge k sorted lists: min-heap of current heads; repeat extract-min and advance same list. Cost $O(N \log k)$ where $N = \sum m_i$.
Heap MCQ: array is valid min-heap iff every parent value $\leq$ both children; no sibling ordering required.

W7 HASHING/TRIES

Hash table: map key to bucket by $h(k)$. Separate chaining stores lists; open addressing probes array. Expected $O(1)$ if load factor controlled and hash distributes well; worst $O(n)$.
Load factor $\alpha = n/N$. Chaining expected successful search $O(1 + \alpha)$; keep $\alpha = O(1)$ by resizing.
Universal hashing: random choice from family reduces collision probability for any fixed key pair; still worst-case insertion sequence can make total cost high, but expected bounds improve.
Pigeonhole collision: if 10000 possible 5-digit/range values map to 100 buckets, some bucket has at least $\lceil 10000/100 \rceil = 100$ values.
Open addressing: linear probing clusters; quadratic/double hashing reduce primary clustering. Must handle deleted slots with tombstones or rehash. Load factor must stay < 1 .

W8 GRAPHS/TRaversal

Graph: $G = (V, E)$, $n = |V|$, $m = |E|$. Directed/undirected, weighted/unweighted, simple/multigraph. Adjacency list space $O(n + m)$; matrix $O(n^2)$, edge test $O(1)$.
BFS: queue, discover by layers from source; gives shortest path distances in unweighted graph. Cost $O(n + m)$ adjacency list. Invariant: when vertex dequeued, its distance is final.
DFS: recursion/stack; timestamps discover/finish; explores deeply. Used for cycle detection, topological order, connected components, SCC, articulation ideas. Cost $O(n + m)$.
BFS vs DFS: BFS good shortest unweighted path/layers; DFS good reachability, topological, backtracking. DFS order not unique without neighbour ordering.
DAG topological order: linear order with every edge $u \rightarrow v$ placing u before v . Exists iff directed graph acyclic. DFS reverse finish time or Kahn indegree queue. SCC: maximal set mutually reachable. Kosaraju: DFS finish order, reverse graph, DFS in decreasing finish; Tarjan uses lowlink stack; both $O(n + m)$.
Graph modification trick: reverse all directed edges to compute distances to target t by running single-source shortest path from t on reversed graph.
Traversal proof: discovered vertices partition into processed/frontier/unseen; each edge inspected once or twice. Parent pointers form BFS/DFS forest.


W9 SHORTEST/MST

Dijkstra: nonnegative weights. Maintain tentative dist, min-priority queue. Extract min u finalises $d[u]$; relax outgoing edges. Binary heap: $O(n + m \log n)$; Fibonacci: $O(m + n \log n)$.
Relax edge (u, v, w) : if $d[v] > d[u] + w$, set $d[v] = d[u] + w$, $\text{parent}[v]=u$, decrease-key.
Dijkstra fail: negative edges can make extracted distance nonfinal. Use Bellman-Ford for negative weights/no negative cycle $O(nm)$.
Feasible edge on $s-t$ path: compute $\text{dist}_s[x]$ from s and $\text{dist}_t[x]$ to t by reversed graph; edge (u, v) feasible under budget σ iff $\text{dist}_s[u] + w(u,v) + \text{dist}_t[v] \leq \sigma$. Scan edges for max weight.
Path via set S : multi-source Dijkstra from all $x \in S$ can compute nearest-set distances; for path $s \rightarrow S \rightarrow t$, combine $\text{dist}_s[x] + \text{dist}_t[x]$ or layered graph for exactly/all most one special vertex.
MST: connects all vertices of undirected weighted graph min total weight. Cut property: lightest edge crossing any cut is safe. Cycle property: heaviest edge on cycle not needed.
Kruskal: sort edges by weight; add if endpoints in different DSU components. Cost $O(m \log m)$; DSU nearly $O(\alpha(n))$.
Prim: grow tree from start; repeatedly add min edge from tree to outside. Heap adjacency list $O(m \log n)$; matrix $O(n^2)$.
Shortest path vs MST: shortest path between two vertices is not necessarily contained in MST; MST minimises total tree weight, not pairwise distances.
Answer style: for graph algorithm design, state representation, preprocessing, main algorithm, output, correctness invariant/cut/relaxation, cost in n, m .
Top final themes: Dijkstra with reversed graph, MST drawing, graph traversal order/layers, SCC/dominator variants. **Dijkstra proof:** let u be extracted min. Any alternative path to u via unsettled x has length at least $d[x] \geq d[u] + w$ nonnegative suffix, contradiction. Hence $d[u]$ final.
MST drawing: sort weights; Kruskal adds edges unless cycle. If equal weights, multiple MSTs possible; state one valid tree and total weight. If graph disconnected, MST does not exist; algorithm returns forest.
DSU: make-set each vertex; find with path compression; union by rank/size. Used in Kruskal and connectivity. Amortised almost constant $O(\alpha(n))$.
Shortest path tree vs MST: Dijkstra parent tree depends on source and preserves shortest distances from source only; MST has no source and minimises total edge weight.

W10 GREEDY

Greedy: make locally optimal choice and prove it can be extended to global optimum. Need greedy-choice property + optimal substructure; otherwise give counterexample.
Interval scheduling: sort by earliest finish; repeatedly choose compatible interval. Exchange proof: earliest finishing chosen interval leaves maximal remaining room. Cost $O(n \log n)$.

W11 INTRO/ASYMPTOTICS

DSA goal: represent data so operations meet required asymptotic bounds; exam answers need **algorithm + correctness + complexity**. State input size (n, m, h, k) , model assumptions, and whether worst/expected/amortised.
O; upper bound: $1 < \Theta(n) \leq O(g) \text{ iff } \exists c, n_0 : f(n) \leq cg(n) \text{ for } n \geq n_0$.
Growth order: $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$.
Constants/lower terms vanish; bases of logs constant factor.
Loop count: arithmetic loop $\sum_{i=1}^n 1 = \Theta(n)$; nested triangular $\sum_{i=1}^n \sum_{j=1}^i 1 = \Theta(n^2)$; halving/doubling loop $\Theta(\log n)$; loop body dependent on n must be summed.
Recurrences: $T(n) = aT(n/b) + f(n)$; compare $n^{\log_b a}$ vs f .
 $T(n) = T(n/2) + O(1) = O(\log n)$; $T(n) = T(n/2) + O(n)$; $2T(n/2) + O(n) = O(n \log n)$; $4T(n/4) + O(n) = O(n^2)$.
Proof patterns: loop invariant (init, maintain, terminate), induction over size/height, exchange argument for greedy, cut/cycle property for MST, divide-and-conquer decomposition, contradiction/counterexample for false greedy.
Pseudo-code style: precise enough to implement; name DS ops and their cost; avoid language-specific syntax if irrelevant. If using sort, heap, BFS/Dijkstra, cite known cost.
Exam traps: $O(n)$ algorithm not guaranteed faster on every finite input than $O(n \log n)$; has expected vs worst-case differs; output size may dominate.

Pattern	Count	Cost
for $i = c$ to n/c	$\Theta(n)$	
while $i \leq 2 \log n$	$\Theta(\log n)$	
all pairs	$n(n-1)/2 = \Theta(n^2)$	
sort/scan	$n \log n + n = \Theta(n \log n)$	

Correctness wording: "Invariant after processing first i items: data structure stores exactly $\dots$ ". At termination $i = n$, invariant implies desired output.
Lower bound reminder: comparison sorting needs $\Omega(n \log n)$ comparisons in worst case; non-comparison sorts exploit bounded alphabet/range/word size.

W2 ADT/LISTS

ADT: specifies operations and semantics, not representation. Implementation choice controls cost: array list, singly/doubly linked list, positional list, map, stack, queue.

Op	Array	Single LL	Double LL
access i	$O(1)$	$O(1)$	$O(1)$
insert front	$O(n)$	$O(1)$	$O(1)$
append	amort $O(1)$	$O(1)$ tail	$O(1)$
remove node	$O(n)$	need prev	$O(1)$

Array list: contiguous, cache-friendly, random access $O(1)$; insert/delete middle shifts $O(n)$; dynamic resize: single resize $O(n)$ but append amortised $O(1)$.
Linked list: stores references; no random access; insertion/removal after known node $O(1)$; more memory overhead; good for frequent local splicing.
Position: abstract handle to node, hides implementation. Validate position belongs to list and is still live; sentinels simplify edge cases.
Reverse-order digit lists: traverse both heads with carry; append digit $(x+y+carry) \bmod 10$; carry $((x+y+carry)/10)$; stop when both null and carry 0. Cost $O(\max(m, n))$.
Sorted vs unsorted list map: unsorted put $O(1)$ if duplicates allowed/append but get/remove $O(n)$; sorted get by linear list still $O(n)$ but supports early stop; array sorted can binary search $O(\log n)$ but insertion $O(n)$.
Iterator mutation: if removing while scanning array, do not skip shifted element. For linked list maintain prev/current.
Space: list nodes $O(n)$, array capacity may be $O(n)$ with unused slack; doubling keeps total copied work $O(n)$ over n appends.
Use cases: queue via linked list with head/tail; stack via dynamic array or linked list; priority queue not ordinary queue: delete-min/max by key.
Invariant for list algorithms: maintain processed prefix result and unprocessed suffix pointer; every node visited once unless nested search.

W3 STACKS/QUEUES/COST

Stack: LIFO, ops push/pop/top in $O(1)$; array stack may resize. **Queue:** FIFO, enqueue rear/dequeue front $O(1)$ with circular array or linked head/tail.
Expression evaluation: two stacks: values and operators. Scan tokens; push nums/ops/ '(' / on ') ' pop op + operands, compute, push result. Fully parenthesised + , × expression cost $O(n)$.
Validity checking: parentheses/XML tags use stack of expected closers/open tags. On closing token, stack top must match; final stack empty. Correctness by nesting invariant.
Deque: insert/delete front/back $O(1)$ with doubly linked list or circular buffer; supports both stack and queue behaviour.
Amortised analysis: sequence cost / operations. Dynamic array doubling: although some pushes cost $O(n)$, total copying over n pushes $O(n)$, amortised $O(1)$.
Remove-even array list: if remove at index shifts left $O(n)$, repeated removals worst $\Theta(n^2)$; linked list with known prev can do $\Theta(n)$ scan.
Complexity from height/depth: tree path algorithm from node to root costs number of ancestors times per-node work. If $\text{bar}(u) = O(\log n)$ and path length h , cost $O(h \log n)$, worst $O(n \log n)$.
If algorithm calls $\text{bar}(u)$ on search path, cost is $\sum_{u \text{ path}} \text{cost}(\text{bar}(u))$, not simply $O(h)$.
ADT selection: Insert-large + delete-min can be queue if inserted keys strictly increasing; append at back, delete front $O(1)$. Heap/BST would be overkill.
MCQ checks: stack reverses order; queue preserves order; priority queue removes by priority, not arrival; circular array full/empty distinguished by size or spare slot.

W4 TREES/RECURSION

Tree terms: root, parent/child, sibling, ancestor/descendant, leaf, depth, height. For node u , subtree T_u , size $|T_u|$, tree height h can be $n-1$ unless balanced.
Traversals: preorder visit-root-left-right; inorder left-root-right (BST sorted); postorder left-right-root; level order BFS queue. All visit each node once: $O(n)$.
Recursive tree analysis: if one recursive call down a path, depth h ; if both children, recurrence sums over subtrees. If work at each node $O(1)$ then $O(n)$. If work $O(|T_u|)$ at every node, worst skew $\Theta(n^2)$, balanced $\Theta(n \log n)$.

W5 BST/AVL/BALANCED

Binary tree implementation: linked nodes with left/right/parent; array heap style only works compactly for complete trees. General/BST array layout wastes space under skew and expensive gaps.
BST property: all keys left $< u$, $key > u$ all keys right. Search/insert/delete follows one root-to-leaf path: $O(h)$. Inorder gives sorted sequence.
Build BST from preorder in $O(n)$: maintain index and allowed range (l, o, h) ; next key belongs if $l < o < h$; create node, recurse left (l, o, x) then right (x, h, h) . Each key consumed once.
Insert/delete in BST: if left child exists, max of left subtree; else climb until first ancestor for which node lies in right subtree. Search for predecessor of value x : track last key $< x$ while descending.
Deletion: leaf remove; one child splice; two children swap/copy predecessor/successor then delete that node. Cost $O(h)$.
BST best/avg if balanced: $O(\log n)$; worst skew: $O(n)$. State $O(h)$ first, then substitute.


W6 MAPS/HEAPS/SORT

Map: key-value dictionary supporting get/put/remove. Implementations: unsorted list, sorted array/list, hash table, balanced BST. Choice depends on ordered queries vs expected $O(1)$.

Impl	get	put	ordered?
unsorted list	$O(n)$	$O(1)$	no
sorted array	$O(\log n)$	$O(n)$	yes
AVL/RB	$O(\log n)$	$O(\log n)$	yes
hash eq	$O(1)$	$O(1)$	no

Priority queue: insert item with key, min/remove-min. Unsorted list: insert $O(1)$ remove-min $O(n)$; sorted list: insert $O(n)$ remove-min $O(1)$; heap: both $O(\log n)$.
Binary heap: complete binary tree with heap-order. Array index: parent $((i-1)/2)$, children $2i+1, 2i+2$ (0-index). Min-heap parent $\leq$ children.
Insert heap: append at end, up-heap/bubble up until heap-order restored; cost $O(n)$. **Delete-min:** replace root by last, down-heap with smaller child; $O(\log n)$.
Heapify: bottom-up downheap from last internal node to root builds heap in $O(n)$, not $O(n \log n)$. Heapsort: heapify + repeated remove $O(n \log n)$, in-place, not stable.
Sorting: selection/insertion worst $O(n^2)$; merge worst $O(n \log n)$ stable extra space $O(n)$; quicksort expected $O(n \log n)$ worst $O(n^2)$; heap sort stable extra space $O(n)$.
Comparison lower bound: any comparison sort worst $\Omega(n \log n)$; counting/radix can be linear if key universe/digit count bounded. Radix on 192-bit coordinates may be large constant/memory and not always practical.
kth smallest: keep max-heap of size k ; scan elements, replace root if new $< \text{root}$; root at end is kth smallest. Cost $O(n \log k)$, space $O(k)$.
Merge k sorted lists: min-heap of current heads; repeat extract-min and advance same list. Cost $O(N \log k)$ where $N = \sum m_i$.
Heap MCQ: array is valid min-heap iff every parent value $\leq$ both children; no sibling ordering required.